

Design Project B Final Project Report

By: Felix Li, Jack Wang, Desmond Loh, Thenuja Wijesuriya

SHA3-256 Keccak-Based Bitcoin Miner

Introduction & Background

Bitcoin mining is the process of finding a numeric value (nonce) which, when combined with a block's header and hashed, yields a hash value below a network-defined target threshold. Miners iteratively increment the 32-bit nonce field in the 80-byte block header and compute the hash until a "valid" nonce is found that produces a sufficiently small hash.

This proof-of-work mechanism makes finding a valid block *computationally hard*, requiring, on average, billions of hash attempts for each block. Traditional Bitcoin uses two rounds of SHA-256 (SHA-2) hashing (double-hash) for each nonce tested. As mining difficulty increased, dedicated hardware emerged to hash extremely fast, making general-purpose processors more outdated.

Why is hashing needed in Bitcoin Mining?

Hashing in Bitcoin serves several important purposes beyond simply finding a nonce. Each block in the blockchain contains a list of transactions, a timestamp, a reference to the previous block's hash, and the miner's chosen nonce. The block header is passed through a cryptographic hash function to produce a fixed-size, unpredictable output.

This hash functions as the block's unique digital fingerprint. The proof-of-work requirement—that the hash must be below a certain target value—forces miners to perform substantial computation before a block can be considered valid. This high computational cost makes it impractical for an attacker to alter the blockchain, since modifying even a single transaction would require re-mining that block and every block that follows it. Through this mechanism, hashing provides security by ensuring that any tampering changes the hash and invalidates the chain. It also enables consensus, as all participants agree on the longest chain representing the greatest amount of accumulated computational work. The Bitcoin protocol further adjusts the target threshold over time, maintaining an average block creation rate of roughly ten minutes regardless of changes in total network hashing power.

Why SHA3-256 (Keccak)?

This project explores the use of Keccak (the core of SHA-3) as an alternative hashing algorithm for Bitcoin mining. Keccak was standardized by NIST in 2015 and differs

fundamentally from SHA-2 in structure. It uses a sponge construction with a 1600-bit state and a permutation-based round function.

Keccak's design is inherently hardware friendly. Its regular structure, reliance on bitwise XOR, AND, and rotation operations, and absence of complex arithmetic enable ultra-fast FPGA implementations and deep pipelining. Prior studies have shown that Keccak can achieve significantly higher throughput than SHA-2 at comparable power consumption. Bertoni et al. also highlight that Keccak's architecture supports efficient parallelism, potentially yielding at least four times the performance of similar SHA-2 hardware designs.

Our Project Scope

In this project, a SHA3-256-based Bitcoin miner is implemented on the Xilinx Kria KV260 platform, utilizing the ARM Cortex-A53 processing system (PS) for control and custom VHDL/AXI-Lite logic in the programmable logic (PL) for hashing operations. The work focuses exclusively on the mining loop, hashing a fixed block header while iterating over the nonce space, allowing performance comparisons between software and hardware implementations without the complexity of a full blockchain node.

Two hardware implementation approaches were explored:

1. Generating the hashing core and control module in Vivado HLS from C/C++ code to produce a fully functional Bitcoin miner.
2. Designing and synthesizing separate VHDL block components manually for direct integration.

Project Motivation & Goals:

The aim of this project is to design a *functional Bitcoin mining system* on a Xilinx Kria KV260 FPGA using SHA3-256 as the hashing algorithm, and to evaluate its performance against standard approaches.

Main Objectives included:

1. Implementing a fully functional SHA3-256 hashing core in hardware (FPGA logic) that can iterate over nonces to mine a block
2. Accelerating hash computation relative to a software implementation, demonstrating the FPGA's capability
3. Analysing the system's performance in terms of throughput (hashes per second), and comparing it to baseline implementations (CPU software and an HLS-based core)

We restrict the scope to the essential mining loop (i.e., hashing a fixed block header with varying nonces). We ignore other blockchain operations (Merkle root calculation, previous block lookup, etc). By focusing on the hashing core, we can measure and evaluate the speed-up from hardware acceleration.

Methodology

VHDL Based Bitcoin Miner

Design Approach:

1. The miner is implemented as a heterogeneous system comprising a hardened processor (Processing System, PS) and programmable logic (PL) on the Xilinx Kria KV260 SoM.
2. The PS, based on an ARM Cortex-A53 CPU, executes software to manage the mining process by providing input data, starting and stopping hash computations, and verifying results. The PL contains the Keccak-256 hashing core and associated control logic, handling the computationally intensive hashing operations.
3. We followed a hardware/software co-design methodology, the heavy-duty hashing is offloaded to a custom VHDL core in the PL, while the PS handles high-level orchestration, such as loading the initial nonce and checking for valid solutions.

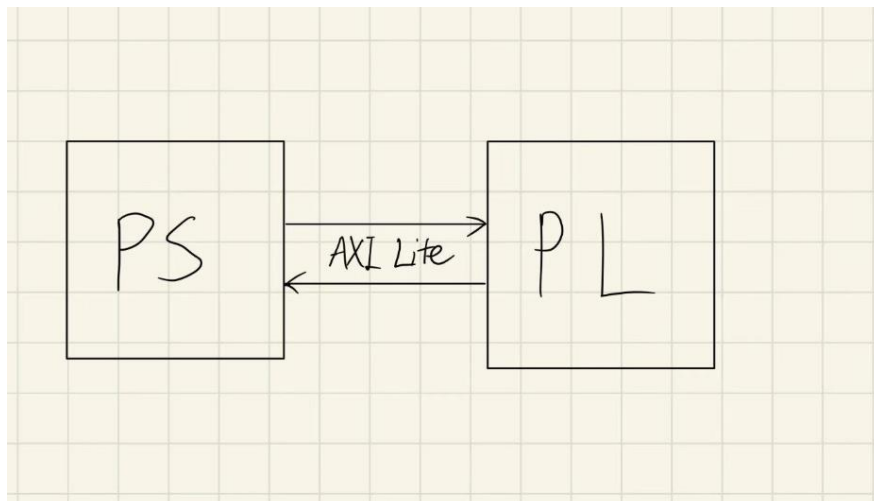


Figure 1: Illustrates the system architecture and workflow. The PS configures the PL core via an AXI4-Lite interface, providing the initial nonce and target threshold before initiating the hashing process. The PL then iterates over nonce values, computes Keccak-256

hashes, and returns results to the PS, which either halts when a valid nonce is found or updates the search parameters.

The PS communicates with the PL through a memory-mapped register interface (AXI4-Lite bus). A small set of 64-bit registers is defined for control and status, including:

- NONCE_START – starting nonce value
- TARGET_LO / TARGET_HI – lower and upper 128 bits of the 256-bit target threshold
- CTRL – start/stop control register
- COMPARE_COUNT – number of hashes tried, with the MSB acting as a “valid nonce found” flag

At runtime, the PS writes the nonce and target values to the designated registers, issues the start command, and enters a polling loop. Every 1 ms, it reads the status register and COMPARE_COUNT to monitor progress and check whether the valid-found flag has been set. If a valid nonce is detected or if the maximum iteration limit is reached, the PS halts the process and retrieves the valid nonce along with the corresponding hash result from the PL.

Hardware Design (PL):

On the programmable logic side, the design was implemented in VHDL as a custom AXI4-Lite IP core connected to the Processing System. The hashing core implements the complete SHA3-256 (Keccak-256) algorithm. This process includes absorbing the 80-byte block header and nonce into the Keccak state, performing the 24 rounds of the Keccak-f permutation, and outputting the resulting 256-bit hash.

In this design, the block header is treated as mostly static, with only the nonce field varying. The Keccak state is a 1600-bit register array that is updated in place during each round’s computations. Once started, the hardware core operates autonomously: rather than requiring the PS to supply each nonce individually, the PL internally increments the nonce and tests hashes in a continuous loop.

The core includes N_reg to store the current nonce and an incrementer that generates the next candidate nonce. The detailed transition progress is as follows,

For each iteration:

- The hash of N_reg (combined with the fixed block header) is computed.
- The resulting 256-bit hash is compared against the target value in T_reg (loaded from the PS) by the comparator unit.
- **If the hash $\geq$ target:** The comparator signals the incrementer to proceed to the next nonce.

- **If the hash < target:**
 - A “valid nonce found” flag is set.
 - The flag is latched into the most significant bit of the COMPARE_COUNT register.
 - The core halts further hashing.
- The valid nonce and corresponding hash result can then be read by the PS via the AXI4-Lite interface.

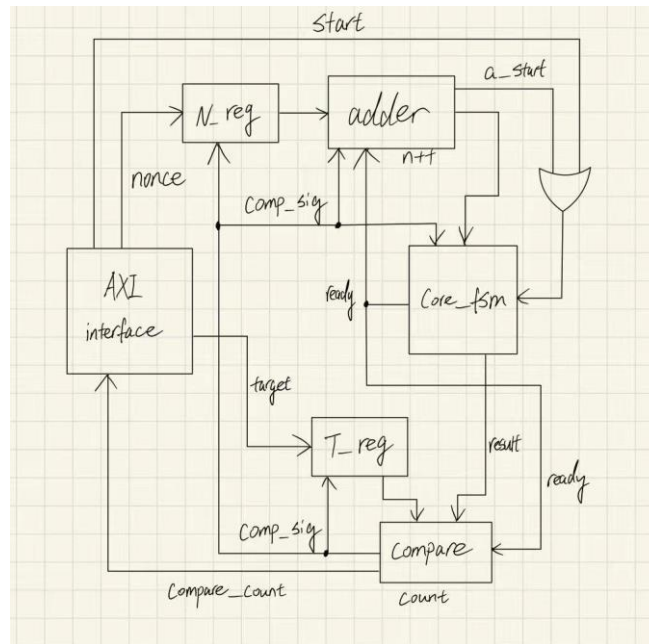


Figure 2: illustrates the internal hardware architecture of the miner core, showing the Keccak hashing core, finite state machine (FSM) for sequencing operations, nonce register and incrementer, comparator, and AXI4-Lite interface logic.

Keccak Core FSM:

- The Hashing algorithm is controlled by an FSM in the VHDL core. The FSM handles input, the execution of the 24 Keccak-f rounds, and output extraction.
- Figure 3 below shows a simplified state transition diagram of the Keccak core's control FSM and sequence of operations

State Transitions:

1. The FSM starts in an **Initial** state, where it resets the 1600-bit Keccak state and waits for the start signal
2. Upon start, it enters **Read_first_input**, loading the initial nonce from **N_reg** and absorbing it (along with the fixed block header) into the state (this involves XORing the 1088-bit rate portion of the state with the input block, since $r = 1088$ for SHA3-256).

3. After absorption, the FSM proceeds to the permutation phase (**St0** through **St1a** states): it triggers the combinational logic that performs the Keccak-f round computations. In our design, each round's logic is split across two pipeline stages (detailed in the next section), so the FSM must wait for the pipeline to complete the full 24 rounds.
4. **The states St1 and St1a represent the pipeline processing:** the FSM signals the start of permutation and then waits until all rounds are done. Once the 24-round permutation is finished, the FSM transitions to **END_HASH1**, where it prepares the hash output.
5. In **END_HASH2**, the final hash value is readied for comparison, and the FSM asserts a ready signal to indicate the hash result is valid. At that point, the Compare unit will latch the result and determine if the hash meets the target value
6. The FSM then either stops (if a valid nonce was found) or resets and returns to **Read_first_input** to process the next nonce (if mining is continuing). If mining continues, the cycle repeats continuously until an external stop or solution is found

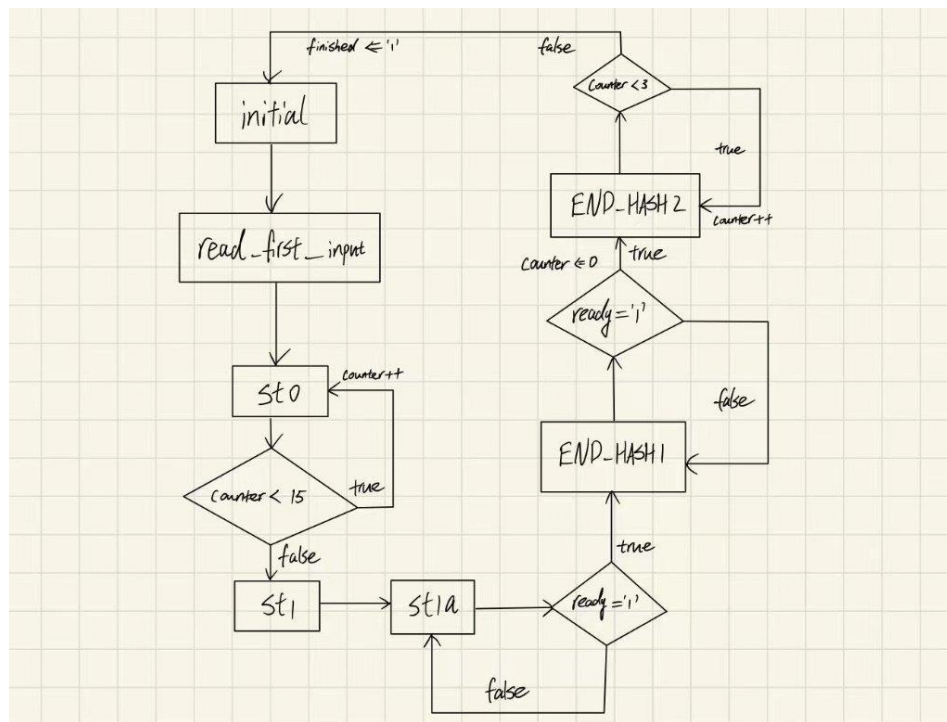


Figure 3: Keccak core FSM and round computations. (θ = theta, ρ = rho, π = pi, χ = chi, ι = iota)

AXI-LITE Interface:

The interface between PS and PL uses Xilinx's AXI4-Lite protocol (a lightweight subset of the AMBA AXI specification) for memory-mapped register access. AXI4-Lite was chosen because the control data did not require high throughput.

- The AXI4-Lite bus consists of five independent channels (address and data for read, address and data for write, plus a write response) and employs a **VALID/READY** handshake on each channel to transfer information reliably
- In our design, the PS acts as the AXI master, initiating read and write transactions to the user logic, which is an AXI slave.
- Internally, the AXI slave logic then sets a start signal to the core FSM. Status updates (like the COMPARE_COUNT) are read by the PS through AXI-Lite read transactions.

Architecture:

The internal architecture shown in figure 2 shows key components of the design:

1. **Nonce Register (N_reg) and Adder:** holds the current nonce and compute the next nonce. N_reg is 32 bits. The Adder increments the nonce by 1 (with wrap-around) upon each new hash trial
2. **Target Register (T_reg):** a 256-bit register (split into two 128-bit registers for AXI access) that holds the target threshold against which computed hashes are compared. This is loaded by the PS at start.
3. **Core FSM (core_fsm):** core fsm that controls the hashing process. It controls when to absorb input vs when to perform permutations
4. **Compare Unit:** a small comparator that takes the 256-bit hash output from the core and compares it with the target value in T_reg. It also maintains a counter of how many hashes have been compared.
 - a. If the hash is **below** the target (meaning a valid solution is found), the Compare unit will assert a comp_sig signal to show that it was successful
 - b. This signal triggers logic to stop the core and latches the found nonce for output.
 - c. The compare unit drives the MSB of the COMPARE_COUNT register (readable by PS) to 1 to indicate a valid nonce was found.
 - d. If the hash is not below target, the compare unit asserts a signal to the Adder to increment and continue to the next nonce, and also increments its internal counter.

HLS-Based SHA-3 Mining

Functional Verification in Software

A reference C implementation (sha3.c) of Keccak-p[1600,24] and the SHA-3/SHAKE API was verified using a dedicated testbench (main.c). The testbench ran official NIST Known Answer Tests (KATs) across SHA-3-224/256/384/512 and SHAKE128/256, ensuring correctness before hardware translation. Performance was also measured to provide a

CPU baseline. These files acted as the model for later verification and were used in Vivado HLS C simulation.

HLS Core Implementation and AXI-Lite Integration

The algorithm was refactored into a synthesizable form (sha3_hls.cpp with sha3_hls.h) by removing unions, using fixed-size arrays, and applying HLS optimisations (loop unrolling, pipelining, array partitioning). The Keccak-f core (sha3_keccakf_hls) was fully parallelised, and a mining pipeline was implemented to hash sequential nonces and compare results to a target. A top-level function (sha3_miner_top) wrapped this pipeline in an AXI-Lite interface with control/status registers (start, stop, status, nonce, target, result, hash_count), allowing direct communication with the Zynq Processing System.

PS-Side Control and Testing

A Processing System control program (miner_ps.c) started and stopped mining, monitored status, read the found nonce, and calculated the hash rate. It interacted with the FPGA via AXI-mapped registers and enforced a timeout to stop if no solution was found.

Results and Evaluation (HLS) :

Sha-3 Hash Function Performance Comparison, **PL clocked at 100MHz**

Implementation	Sustained Write (MH/s)	Time for 1M Hash	Power Usage (W)
SHA-3 Miner (VHDL)	-	-	2.141
Tiny-sha3 (HLS)	6.281	0.256s	2.422
Tiny-sha3 (CPU: ARM A53 - KV260)	0.828	1.315s	4.155
Tiny-sha3 (CPU: R9 5900HS)	1.595	0.611s	12.54

The performance data shows a clear advantage for the HLS-based SHA-3 hashing speed compared to CPU-only implementations. The Tiny-sha3 HLS design achieved **6.281 MH/s**, completing 1 million hashes in **0.256 seconds** at a low power draw of **2.422 W**, offering the best efficiency among all tested platforms.

The ARM Cortex-A53 CPU on the KV260 board delivered only **0.828 MH/s** (1.315 seconds per 1M hashes) and consumed more power (**4.155 W**) than the FPGA implementation, highlighting the benefits of hardware acceleration. The high-

performance R9 5900HS CPU achieved **1.595 MH/s** with 0.611 seconds per 1M hashes but at a much higher power usage (**12.54 W**).

The manual VHDL miner could not be evaluated for performance due to functional issues, but its power profile (**2.141 W**) indicates potential for competitive efficiency if debugging resolves the FSM and PS communication problems.

Sha-3 miner performance evaluation:

Miner Implementation	Sustained Hash Cycle (kH/s) (set target to 0x1)	Time taken for a target of 3E9+5	Power Usage (W)
Tiny-sha3 (PS-HLS)	174.2	0.51s	6.454
Tiny-sha3 (CPU: ARM A53 - KV260)	67.6	0.91s	5.116
Tiny-sha3 (CPU: R9 5900HS)	119.5	0.33s	11.76 (?)

It was expected that with the control logics of the above diagram added, the average hash rate would decrease, but the actual performance degradation was surprisingly huge. Both CPU approaches' have its sustained hash speed axed (**x86 by x13.4 slowdown, Kria's CPU by x12.2 slowdown**). The **PS-PL approach has the largest slowdown of x36.0**. Its larger slowdown could be attributed to constant AXI-LITE control signal exchange as the **PS code both controls and benchmarks** the PL performance through AXI for each clock cycle, if AXI-STREAM was used we would likely observe a lesser slowdown from pure hashing results.

Also, for some reason the power usage of the x86 miner somehow consumed less power than just pure hashing. It's not yet known exactly why this behaviour would occur, but it could be attributed to how the OS power-balances the miner process against background tasks.

Discussion:

This project developed a SHA3-256 Bitcoin miner on the Xilinx Kria KV260 using two approaches: a Vivado HLS-generated design and a fully manual VHDL design. The HLS version, implementing both the hashing core and control module in C/C++, was successfully synthesised, integrated into the block design, and verified as a fully functional miner.

The manual VHDL version was also designed, synthesised, and integrated, with the block design and bitstream successfully generated. However, it is not yet fully functional due to issues in the FSM logic and Processing System (PS) communication, which prevented correct mining operation by the project deadline.

Testing of the HLS miner showed that moving the hashing loop into FPGA logic provided a measurable performance improvement over a PS-only software miner. Throughput was still limited by the single-core architecture, AXI-Lite control overhead, and the achievable clock frequency, indicating opportunities for further optimisation.

Conclusion:

The project achieved its main objective of implementing a functional SHA3-256 FPGA-based Bitcoin miner and comparing two hardware design flows. The HLS-based miner was fully operational and demonstrated the benefits of hardware acceleration. The manual VHDL version, while fully designed and integrated, remains non-functional due to FSM and PS communication issues.

If more time had been available, we would have focused on debugging the AXI interface and FSM logic in the fully VHDL implementation to achieve correct operation. Future work should also explore deeper pipelining, multi-core parallelism, and high-bandwidth PS-PL data transfer methods such as DMA to further improve performance.

Bibliography & References:

Bertoni, G., Daemen, J., Peeters, M., & Van Assche, G. (2011). *The Keccak SHA-3 submission – Main Document*. (Keccak design document)

Walker, G. (2024). *Nonce – Field in the Block Header used for Mining*. Learn Me A Bitcoin (blog).

Investopedia (2024). *Nonce: What It Means and How It's Used in Blockchain*. (Explains the role of the nonce in Bitcoin mining)

Xilinx (2020). *AXI Reference Guide UG761*. (AXI4-Lite protocol details for memory-mapped interfaces)

Sideris, A., Sanida, T., & Dasygenis, M. (2023). “A Novel Hardware Architecture for Enhancing the Keccak Hash Function in FPGA Devices.” *Information*, 14(9), 475.
(Discusses pipelining and optimizing SHA-3 on FPGAs).